

오픈소스SW의 이해

CHAPTER 1

SEMESTER | 2025-2

박성미

목차

Table of Contents



01. 오픈소스 소프트웨어란?

02. 오픈소스 소프트웨어의 역사

03. 오픈소스 소프트웨어 개발

04. Version Control System

The background is a photograph of a university campus, identified by the text as Hanlim University. It shows a large, modern building with a glass facade, surrounded by greenery and a parking lot with several cars. The entire image is covered with a semi-transparent blue filter. The text "오픈소스 소프트웨어란?" is written in white, bold Korean characters across the middle of the image.

오픈소스 소프트웨어란?

한림대학교

1. 오픈소스 SW란?

1 오픈소스의 정의

- 누구나 자유롭게 접근하여 활용, 수정, 배포할 수 있는 코드

“미래의 **협업**을 위해 소스코드가 **공유**되어야 함”

- 오픈소스 SW 특징:

- ① 공개적으로 접근 가능한 소스코드
- ② 자유롭게 확인, 수정, 제작, 재배포 가능
- ③ 저작권자의 권익을 보호
- ④ 일반 대중의 공동 연구로 시험, 성능 개선



Source (코드)가
Open(공개)된
소프트웨어!



버전 관리
소프트웨어



저장소+협업
웹호스팅 서비스

1. 오픈소스 SW란?

2 오픈소스 vs 비공개 vs 상용

* 듀얼라이선스: 소프트웨어를 둘 이상의 조항/조건으로 배포

반대개념

구분	오픈소스 소프트웨어 (Open Source Software)	비공개 소프트웨어 (Proprietary Software)	상용 소프트웨어 (Commercial Software)
저작권	존재(저작권 없는게 아님!)	존재	존재
소스코드 공개 여부	일반적으로 소스코드 공개	일반적으로 소스코드 비공개	일반적으로 비공개, 그러나 일부 오픈소스로 제공 가능
사용자 권리	자유롭게 사용, 복제, 배포, 수정, 활용 가능	사용자가 소프트웨어를 자유롭게 사용, 복제, 배포, 수정, 활용하는 것이 제한 됨	사용 권한은 일반적으로 유료 이며, 사용에 제한이 있음
라이선스	오픈소스 라이선스 적용, Copyleft 가능	독점적 라이선스 적용, Copyright 주장	주로 유료 라이선스 적용, 듀얼 라이선스 가능
수익 모델	주로 무료로 제공, 서비스나 지원에서 수익 창출	소프트웨어 자체 판매로 수익 창출	제품 판매 및 서비스로 수익 창출, 듀얼 라이선스* 가능
특징	자유로운 사용과 배포를 통해 커뮤니티 중심으로 발전	소프트웨어 사용 및 배포가 제한적	수익 창출 목적, 일부는 오픈소스로 제공 가능

1. 오픈소스 SW란?

2 OX 퀴즈

클라썸 챗으로!

문장	정답
무료 소프트웨어는 모두 오픈소스이다	
GitHub에 코드가 공개되어 있으면 무조건 오픈소스이다	
오픈소스에도 저작권이 있다	
오픈소스는 유료로 판매할 수 없다	

1. 오픈소스 SW란?

2 오픈소스 vs 비공개 vs 상용

...왜 소스코드를 공개하는 걸까?

오픈소스 생태계 맛보기

한림대학교

오픈소스 생태계 맛보기

- (1) FigJam 로그인 하기
- (2) 팀 중 1명이 FigJamBoard 공유
- (3) 다음 주제 중 하나를 선택해서 조사 및 맵핑
 - (1) 오픈소스 프로젝트 자료 정리 해보기:
 - 무엇을 해결하는가
 - 오픈소스라고 판단한 증거
 - 누가 만들고 지원하는가
 - 가장 의외였던 사실
 - ① 우리가 이미 사용해 본 오픈소스 찾기
 - ② 돈을 벌고 있는 오픈소스 프로젝트 찾기
 - 소속제단(있는 경우)
 - ③ 코드 외에 번역·문서·디자인으로 기여하는 프로젝트 찾기
 - 라이선스 (오픈소스 구분!!)
 - ④ 여러 오픈소스를 조합하여 만든 서비스 찾기
 - ⑤ 코드·데이터·모델 중 일부를 공개한 SI 프로젝트 찾기
 - 경쟁관계
 - 공식 페이지 또는 저장소 링크



제품 ▾ 솔루션 ▾ 커뮤니티 ▾ 리소스 ▾ 가격

FigJam

함께 만드는 더 나은 디자인. 팀과 함께 아이디어를 탐색하고, 실감 나는 프로토타입을 제작하며, 더 효율적으로 제품을 개발해 보세요. FigJam은 모든 요금제에 포함되며, 모든 시트에서 사용할 수 있습니다.

FigJam 무료 시작

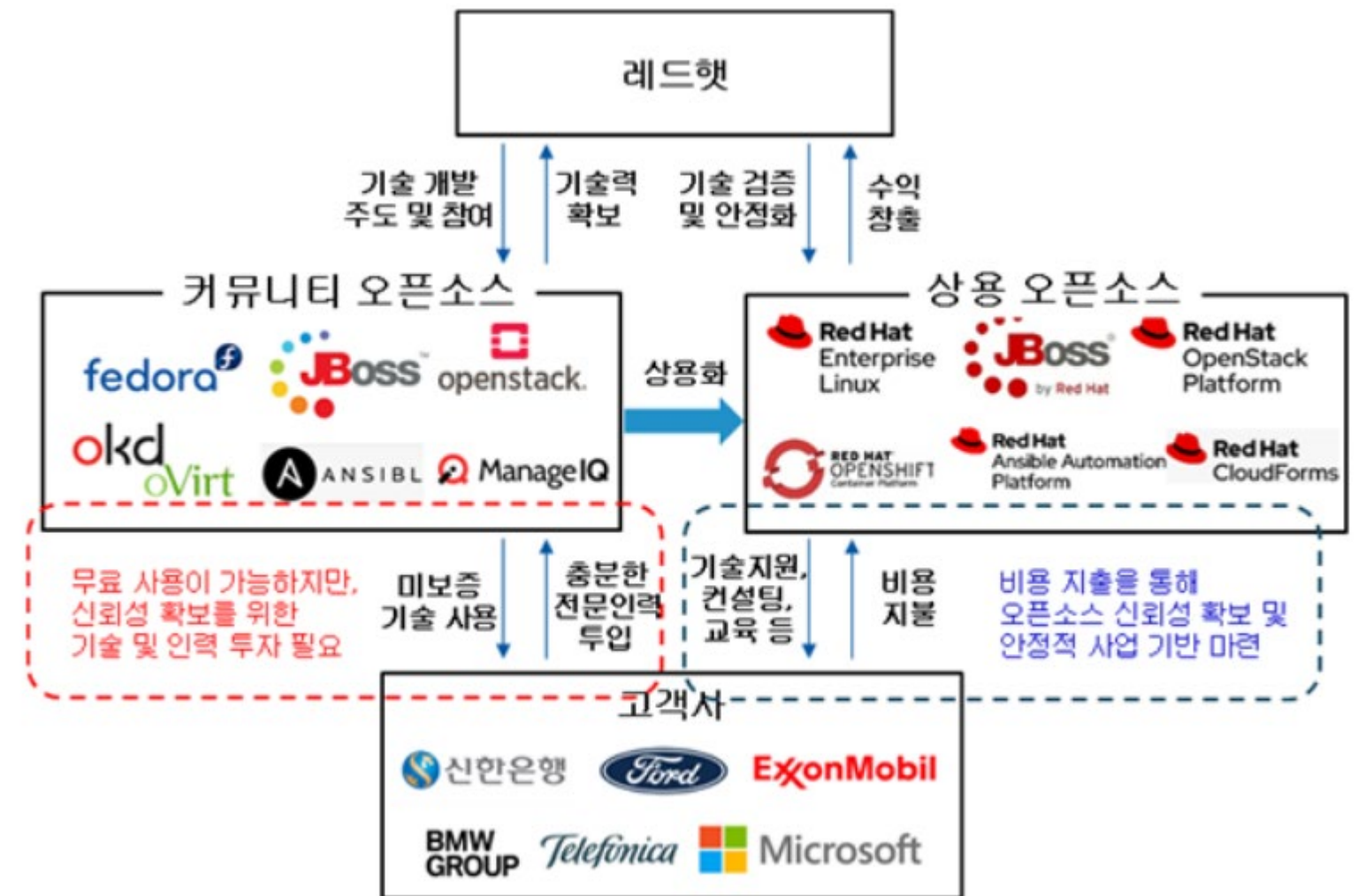
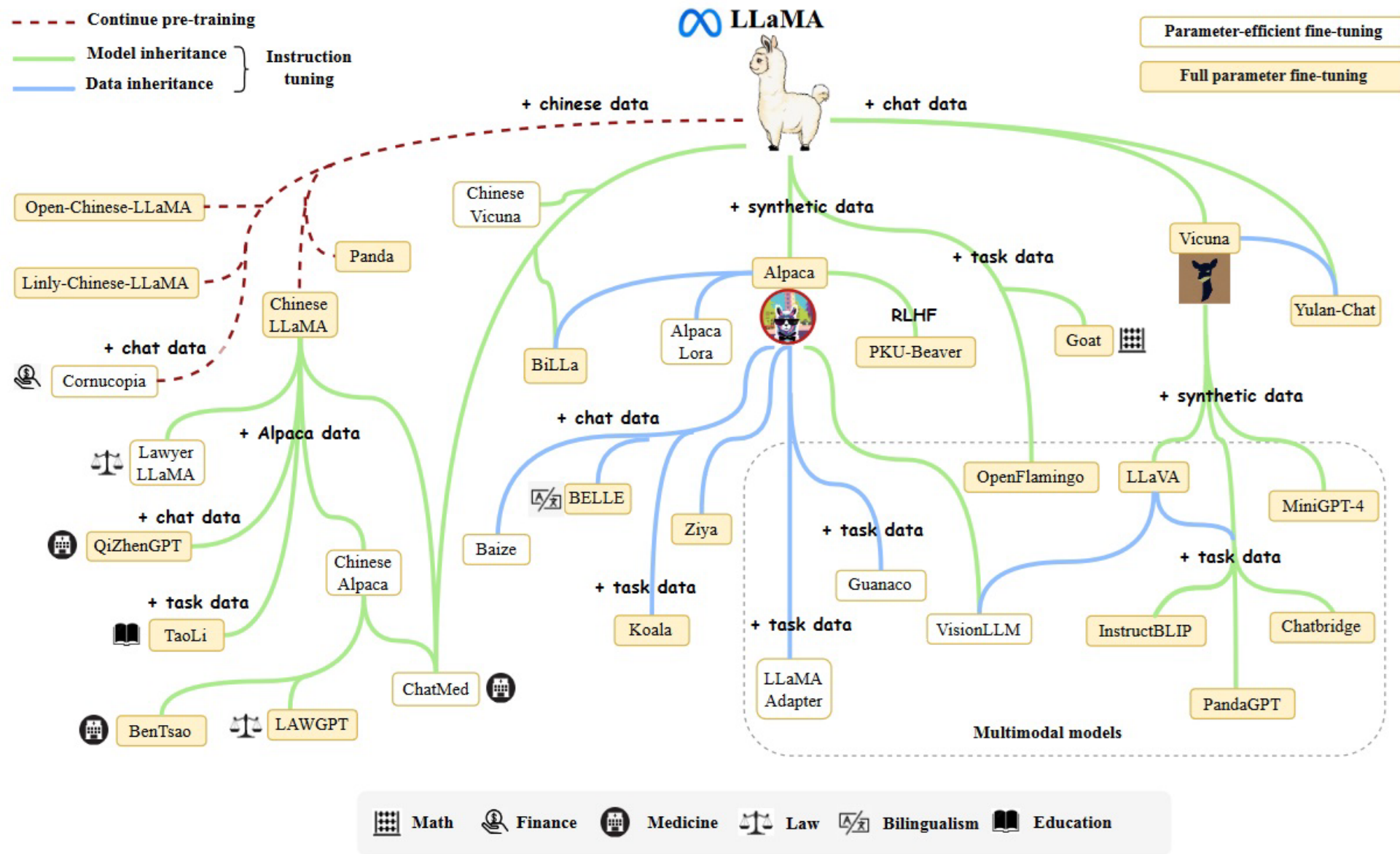
영업팀에 문의

오픈소스 생태계 맛보기

오픈소스 프로젝트 자료 정리 해보기:

- ✓ 무엇을 해결하는가
- ✓ 오픈소스라고 판단한 증거
- ✓ 누가 만들고 지원하는가
- ✓ 가장 의외였던 사실
- ✓ 소속재단(있는 경우)
- ✓ 라이선스 (오픈소스 구분!!)
- ✓ 경쟁관계
- ✓ 공식 페이지 또는 저장소 링크

오픈소스 생태계 맛보기



오픈소스 소프트웨어의 역사

한림대학교

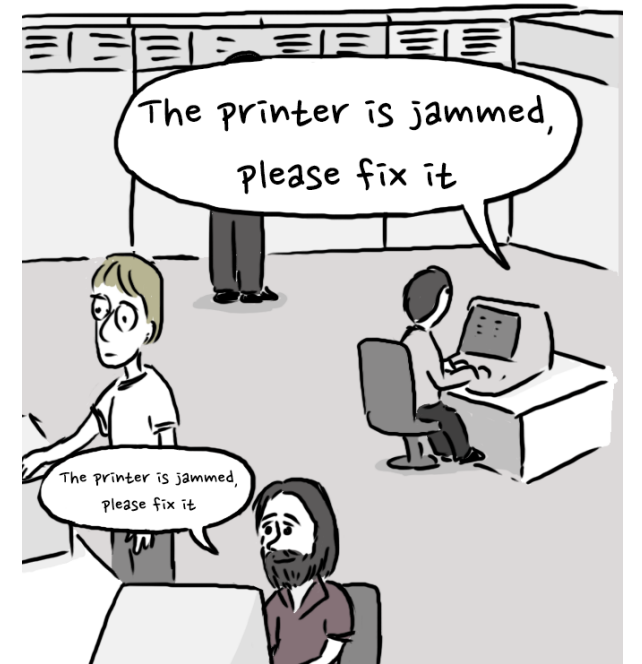
2. 오픈소스 SW의 역사

1 리처드 스톨만 (Richard Matthew Stallman)

- 1970년 후반, RMS은 MIT 인공지능 랩에서 연구원으로 근무
 - 연구소 프린터가 자주 고장 → 모든 터미널로 알림 메시지가 가도록 프로그램 작성
- 프린터 제어 프로그램의 소스코드를 얻기 위해 개발자 컨택
 - 하지만, 개발자는 NDA에 서명하고 개발하여 소스코드를 공유 불가
- 소프트웨어를 사용하려면 **비용 지불, 자료 유출 불가, 수정 불가 등 제약사항 발생**
- “Free Software”** 용어의 탄생 (상품화되는 소프트웨어에 반작용적으로 등장)



<https://joone.net/>



It all started with a laser printer...



2. 오픈소스 SW의 역사

2 프리 소프트웨어 (Free Software)와 GNU 선언문 (GNU Manifesto)

“Free Software as in ‘FREE SPEECH’ NOT as in ‘FREE BEER’!”

자유, 권리

무료 서비스



2. 오픈소스 SW의 역사

2 프리 소프트웨어 (Free Software)와 GNU 선언문 (GNU Manifesto)

“Free Software as in ‘FREE SPEECH’ NOT as in ‘FREE BEER’!”

자유, 권리



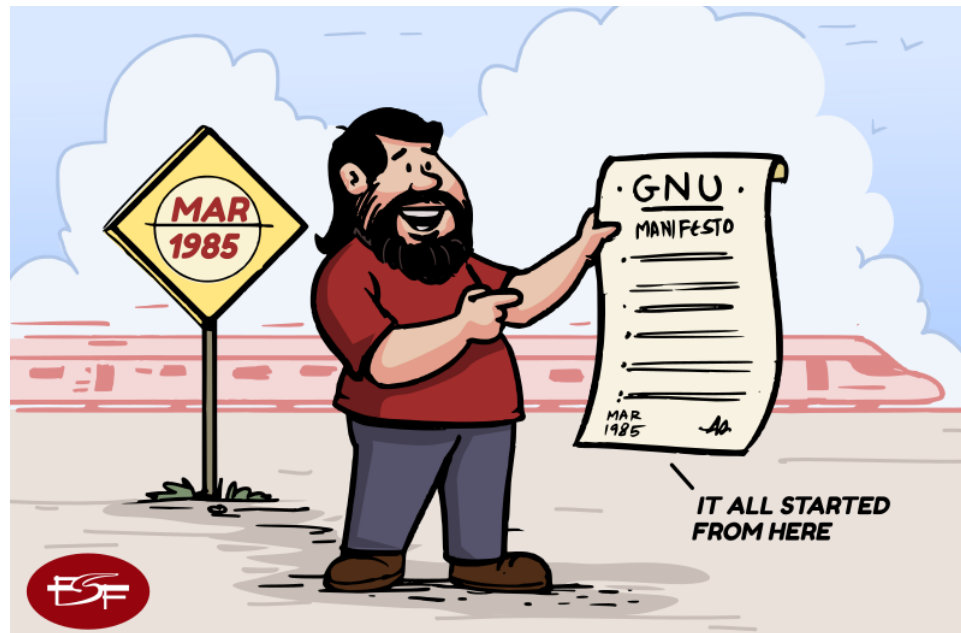
무료 서비스



2. 오픈소스 SW의 역사

2 프리 소프트웨어 (Free Software)와 GNU 선언문 (GNU Manifesto)

GNU 선언문의 4가지 자유(Freedom)



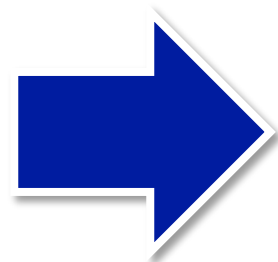
<http://www.gnu.msn.by/gnu/manifesto.old.ko.html>

- Freedom to run the program **as you wish**
(프로그램을 **실행**할 수 있는 자유)

- Freedom to **help your neighbor**. That's the freedom to redistribute the exact copies of the software when you wish (프로그램을 **재배포**할 수 있는 자유)

- Freedom to **study** the source code of the program and then **change it** so the program does what you wish
(프로그램을 **채택하고 연구**할 수 있는 자유)

- Freedom to **contribute to your community**. That's the freedom to distribute copies or modified versions when you wish
(프로그램을 **개선**할 수 있는 자유)



위 4가지를 충족해야만 자유 소프트웨어 (Free Software)

2. 오픈소스 SW의 역사

3 GNU 프로젝트의 탄생



<https://joone.net/>

- 운영체제 (컴퓨터 소프트웨어 모음집)
- GNU는 "GNU's Not Unix!"의 약자로, Unix의 상업적 제약에서 벗어나 자유로운 대안을 제공
- GNU의 목표는 Unix와 유사하지만, 모든 사용자가 **자유롭게 사용할 수 있는 운영체제**를 만드는 것
- GNU 프로젝트에서 탄생한 주요 도구들: GNU Emacs(텍스트 편집기), GCC(컴파일러), GDB(디버거), Bash (셸)
 - GNU Emacs 개발 당시 gmacs(Gosling Emacs)의 코드를 빌려왔지만 회사에서 소송 → 모든 것을 자체 개발
 - 결국, 1985년 GNU Emacs를 개발하여 GNU의 가능성을 보여줌



2. 오픈소스 SW의 역사

3 GNU 프로젝트의 탄생



❖ GNU의 가장 핵심적인 성과:

- 자유 소프트웨어
- 문서화
 - 문서화가 충분히 개발되지 않으면 소프트웨어는 사용하기 어려움
 - 딕 카핀스키 교수 (샌프란시스코 대학교): GNU 프로그램 작성시 1,000불의 상금을 내걸음
 - 프리 코드 작성 외 부수적 업무의 중요도를 알게 됨
- 라이선스
 - 당시 미국 연방정부는 1974년 '저작권이 있는 작품의 신기술적 사용에 관한 위원회'에서 컴퓨터 프로그램의 저작권을 인정 → GPL (GNU General Public License)을 발표하는 계기가 됨
 - 카피레프트(Copyleft)의 용어 등장
 - 1989년: GPL 1.0 발표
 - 1991년: LGPL (GNU Lesser General Public License) 발표

※ 사용자가 다른 사람과 프로그램 복사본을 공유할 수 있으며,
어떻게 합법적으로 코드를 수정하고 소프트웨어를 제작할 수 있는지
명시

2. 오픈소스 SW의 역사

4 희망의 커널: 리눅스

❖ 리눅스 커널의 개발

- 리누스 토르발즈가 개인 프로젝트로 리눅스 커널을 개발
- 리눅스 커널은 자유롭게 사용할 수 있는 유닉스 계열의 커널로, GNU 프로젝트와의 결합을 통해 완전한 운영체제를 구성할 수 있게 됨
- 타넨바움의 미닉스가 (1) 유료이며, (2) 기술적 한계가 있다는 것에 불만
➔ 터미널 에뮬레이터부터 시작하여 운영체제 커널 개발
- 1991년, 8월 유즈넷 게시판을 통해 **커널 프로젝트 발표**

2. 오픈소스 SW의 역사

4 희망의 커널: 리눅스

미닉스를 쓰는 애들아 모두 안녕---

나는 386(486) AT 복제품용으로 (무료) 운영체제를 만들고 있어(이건 그냥 취미생활이야, gnu처럼 거대하고 전문적인 것은 아니야). 4월부터 서서히 진행해왔고 이제 쓸 수 있는 상태가 거의 되어 가고 있어.

나는 사람들이 미닉스에서 좋아하는 점과 싫어하는 점이 무엇인지 피드백을 받았으면 해. 왜냐하면 내 운영체제는 조금은 그걸 닮았거든 (다른 것들 중에서도 (현실적인 이유 때문에) 파일 시스템의 물리적인 배치가 똑같아).

현재는 bash(1.08)와 gcc(1.40)를 이식한 상태이고 제대로 돌아가는 것 같아. 그러니까 몇 달 안에 뭔가 실제로 쓸만한 걸 가지게 될 것 같다는 말이지. 그래서 대부분 사람들이 원하는 기능이 무엇인지를 알았으면 해. 아무 제안이라도 환영. 하지만 그걸 구현한다고 약속은 못하겠는데 :-)

리누스 (torvalds@kruuna.helsinki.fi)

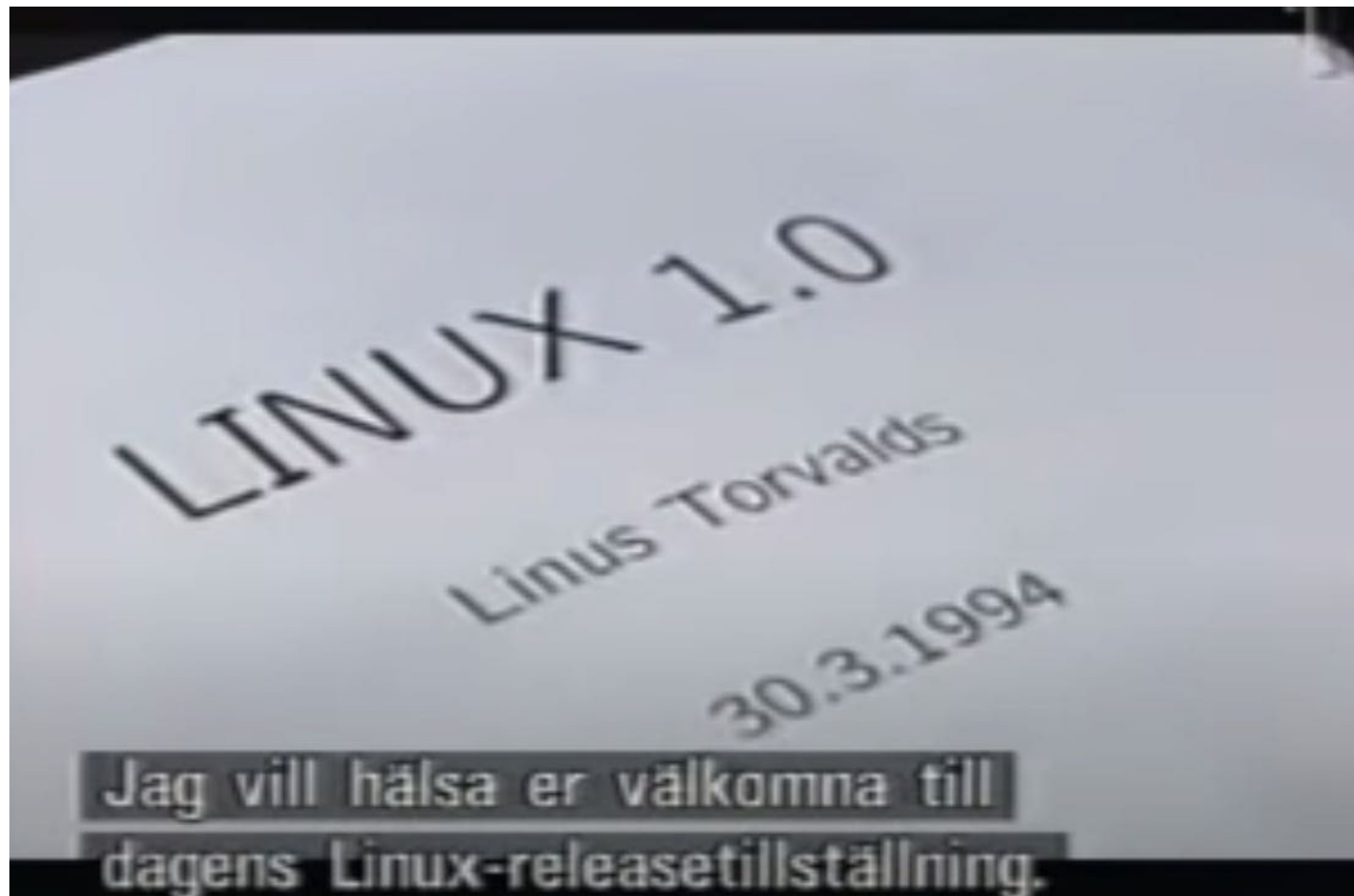
추신) 맞아 - 여기에 미닉스 코드는 하나도 들어 있지 않아. 그리고 멀티스레드를 지원하는 파일 시스템을 가지고 있지. 이건 '전혀' [이식성]이 없어(왜냐하면 386 태스크 스위칭 등등을 쓰고 있거든). 그리고 아마도 AT 하드디스크가 아닌 것은 지원하지 못할 거야. 내가 가진 것이 그거밖에 없어서 말이지. :-⁽⁵²⁾

2. 오픈소스 SW의 역사

4 희망의 커널: 리눅스

❖ 리눅스 커널의 개발

- 당시 리누스 토르발스는 인터넷을 통해 소프트웨어를 배포하는 방식 선호
 - 실제로 리눅스(Linux) 소스 코드는 헬싱키 대학 FTP 서버에 올려 배포
- 1993년 네트워킹이 지원되는 리눅스 커널 개발 완성 → 리눅스 1.0 발표

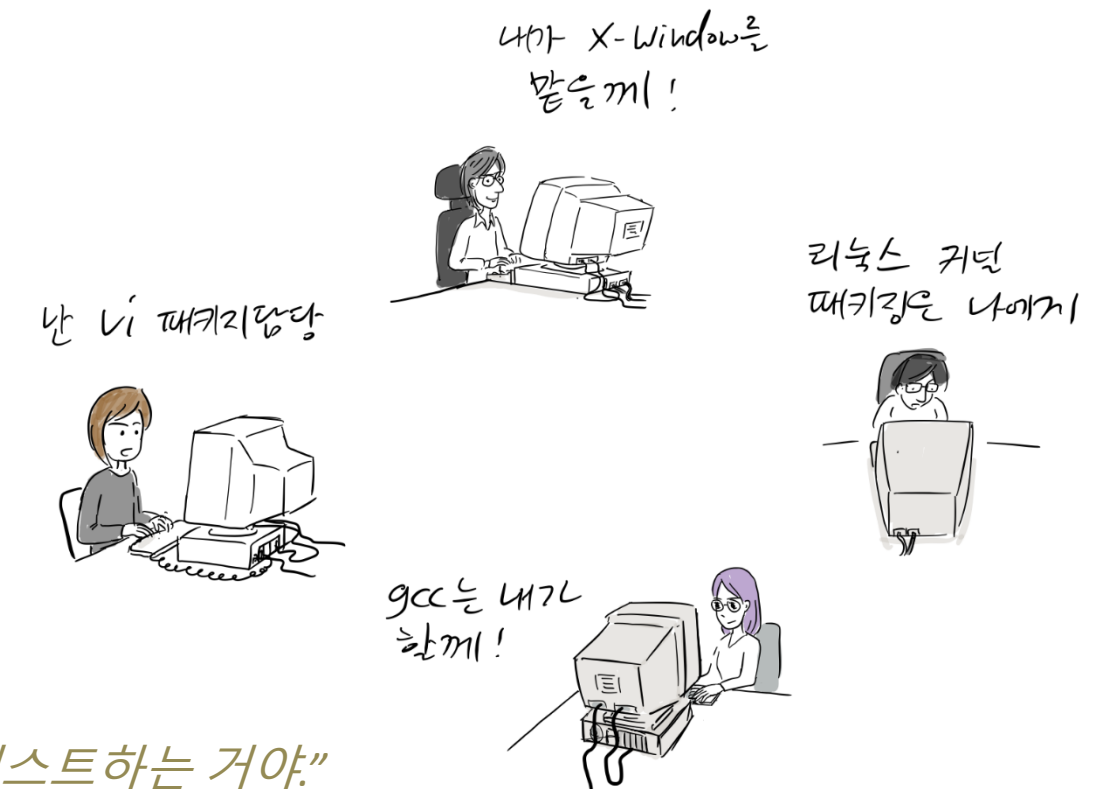


2. 오픈소스 SW의 역사

4 희망의 커널: 리눅스

❖ 리눅스 커널 성공의 비결

- 라이선스
 - GPL을 채택하면서 해커의 가치를 키우는 것이 목표였던 FOSS 진영의 지원을 받게 됨
- 무료
 - BSD 라이선스 기반 운영체제도 무료로 얻을 수 있었으나, 공식적 복사본은 유려였음
 - 초기부터 무료였던 리눅스와 대조적
- 커뮤니티
 - 개발자 커뮤니티가 자발적으로 프로젝트에 동참, 협업 중심
 - 하나의 기관이 아닌 분산형 개발 모델
 - ➔ 2008년 깃헙(Github) 공개되며 열린 협업이 강조



"개발 공동체는 모두 인터넷에 연결되어 있고,
각자 하나의 패키지를 맡아 릴리스하고 사용자가 테스트하는 거야."

2. 오픈소스 SW의 역사

5 성당과 시장 모델

❖ 성당과 시장 (The Cathedral and Bazaar)

성당과 시장 – 어떤 의미일까?

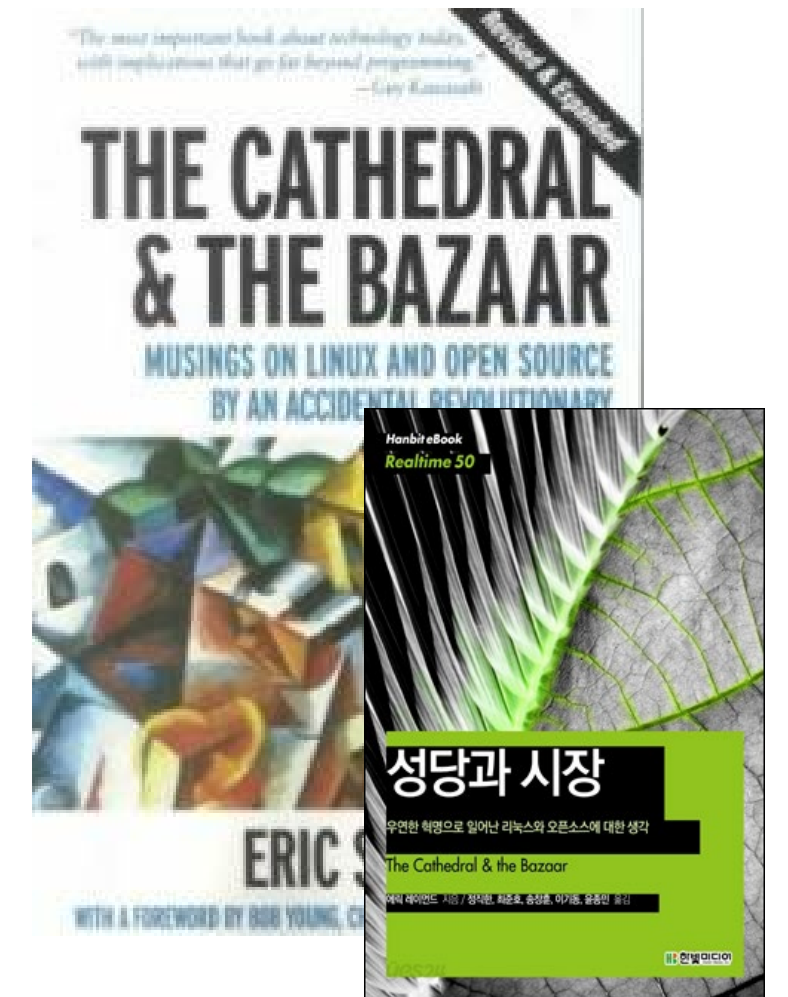


2. 오픈소스 SW의 역사

5 성당과 시장 모델

❖ 성당과 시장 (The Cathedral and Bazaar)

- 오픈 소스 소프트웨어 개발 방식을 설명하는 두 가지 대조적인 방식
 - 에릭 레이먼드(Eric S. Raymond)가 " **성당과 시장(The Cathedral and the Bazaar)** " 에서 처음 소개
 - 1997년 리눅스 커널 개발 회의에서 발표 → 오픈 소스 소프트웨어 철학을 정립하는 데 중요한 역할
- 에릭 레이먼드
 - 오픈 소스 소프트웨어 운동의 주요 인물 중 한 명
 - 자유 소프트웨어와 오픈 소스 개발의 가치를 강조 → 상업적 소프트웨어 개발 방식과의 차이점을 논의
 - 리눅스 커널의 성공을 오픈 소스 모델의 강력한 증거로 사용
- 성당과 시장
 - **성당 모델**은 전통적인 소프트웨어 개발 방식을 나타내며, **폐쇄적이고 중앙집중적인** 구조
 - 개발은 제한된 팀에서 이루어지며, 일반 사용자는 최종 릴리스 이후에만 소프트웨어를 볼 수 있음
 - 예) GNU 프로젝트의 GCC, GNU Emacs
 - **시장 모델**은 리누스 토발즈가 리눅스 개발에서 사용한 분산형 협업 방식을 기반으로 하며, 개방적이고 유연한 개발 구조
 - 모든 사람이 코드에 기여할 수 있으며, 코드 수정이 즉각적으로 공개됨
 - 예) 리눅스 커널



2. 오픈소스 SW의 역사

5 성당과 시장 모델

❖ 성당 모델 vs 시장 모델

구분	성당 모델 (GNU 프로젝트)	시장 모델 (리눅스 커널)
개발 환경	폐쇄적인 개발 환경	개방된 개발 환경
참여자	소수의 개발자들이 소스 코드를 관리하고 외부인에게 제한된 접근 허용	모든 사람이 자유롭게 소스 코드에 접근하고 기여할 수 있음
의사결정 구조	중앙집중식 의사결정 (개발 리더가 모든 결정을 내림)	분산된 의사결정 (개발자들이 자율적으로 문제 해결 및 기여)
릴리스 주기	완성된 버전만 릴리스 릴리스 주기가 길며 안정적인 버전 제공	"자주 릴리스, 자주 배포" 원칙을 따름 릴리스 주기가 짧고 유동적
릴리스 방식	완성된 이후에만 코드가 공개 중간 과정은 외부에 공개되지 않음	코드가 항상 공개되어 있음 개발 중에도 커뮤니티에서 실시간으로 볼 수 있음
예시	GNU Emacs, GCC (폐쇄적 개발과 안정적 릴리스를 강조)	리눅스 커널 (분산형 협업, "자주 릴리스, 자주 배포" 원칙 채택)
대표 인물	GNU 프로젝트	리누스 토발즈 (리눅스 창시자, 시장 모델 제시)

2. 오픈소스 SW의 역사

<https://joone.net/>

6 Free Software? Open Source? FOSS?

- 1998년 초: 넷스케이프 소스 코드 공개 → 기업용 모델 논의
- 1988년: 'Free Software'라는 용어가 '공짜'와 '자유'를 혼동하게 할 수 있다고 지적, 'Open Source'라는 용어를 제안 → Free & Open Source Software (FOSS) 개념 정의
- 국내에서는 자유 소프트웨어와 오픈소스 소프트웨어를 통합하여 "오픈소스SW"로 표기함



2. 오픈소스 SW의 역사

7 Open Source Initiative

(오픈소스SW 라이선스 가이드, 2024)

OSI의 Open Source 프로젝트 요구사항 10 가지:

1) 자유 배포 (Free Redistribution):

소프트웨어의 재배포에 제한이 없어야 하며, 별도의 라이선스 비용을 요구할 수 없다.

2) 소스코드 공개 (Source Code Open):

소스코드는 반드시 공개되어야 하며, 프로그램의 수정 및 배포가 가능해야 한다.

3) 2차적 저작물 허용 (Derived Works):

원저작물을 개작한 2차적 저작물의 생성과 배포가 허용되어야 한다.

4) 소스코드 수정 제한 (Integrity of The Author's Source Code):

소스코드 수정이 제한될 수 있으나, 수정된 프로그램의 자유로운 배포는 허용해야 한다.

5) 개인이나 단체에 대한 차별 금지 (No Discrimination Against Persons or Groups):

라이선스는 모든 개인과 단체에 차별 없이 동일하게 적용되어야 한다.



2. 오픈소스 SW의 역사

7 Open Source Initiative

(오픈소스SW 라이선스 가이드, 2024)

6) 사용 분야에 대한 제한 금지 (No Discrimination Against Fields of Endeavor):
특정 분야에 대한 소프트웨어 사용을 제한해서는 안 된다.

7) 라이선스의 배포 (Distribution of License):
라이선스는 모든 사용자에게 동일하게 적용되며, 추가 승인이나 양도 과정이 필요 없다.

8) 라이선스 적용상의 동일성 유지 (License Must Not Be Specific to a Product):
라이선스는 특정 제품에만 적용되지 않고, 모든 배포 단계에서 동일한 효력을 가져야 한다.

9) 다른 라이선스의 포괄적 수용 (License Must Not Contaminate Other Software):
오픈소스와 함께 배포되는 다른 소프트웨어에 제한을 두어서는 안 된다.

10) 라이선스의 기술적 중립성 (License Must Be Technology-Neutral):
라이선스 조항은 특정 기술이나 인터페이스에 국한되지 않고, 모든 기술에 동일하게 적용되어야 한다.



2. 오픈소스 SW의 역사

8 FOSS와 비즈니스

❖ GNU/Linux의 탄생

- GNU의 도구들과 리눅스 커널의 결합으로 완전한 오픈소스 운영체제인 GNU/Linux가 탄생
- 이 결합은 오픈소스 소프트웨어의 발전에 중요한 역할을 하였으며, 오픈소스 커뮤니티의 확장을 가속화

❖ 오픈소스 운영체제

- 1993년: Red Hat이 설립되어 리눅스 배포판을 제공
- 운영체제는 무료, 그에 따른 운영/서비스를 상업화하여 상업적인 오픈소스 소프트웨어의 가능성을 보여줌 (2018년 IBM에 인수)
- 2007년: Google 안드로이드 운영체제를 무료 공개
- 2008년: 안드로이드 오픈소스 선언 – Apache 라이선스로 공개

2. 오픈소스 SW의 역사

9 LAMP 스택

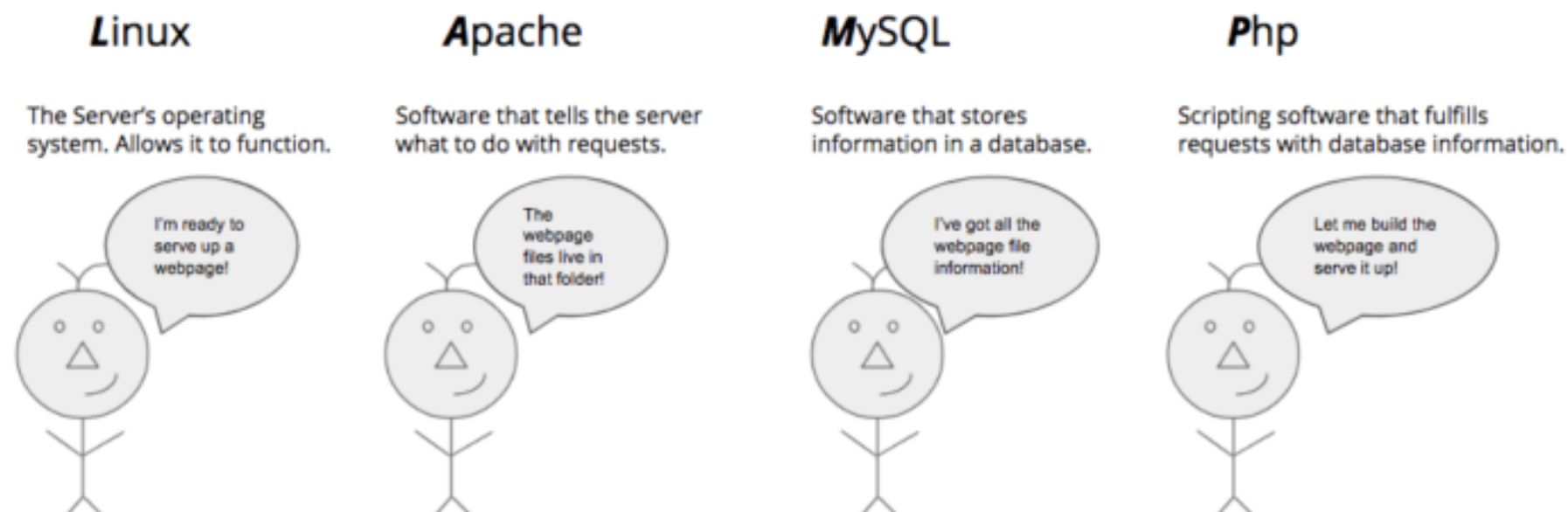
- ❖ LAMP는 오픈 소스 철학을 실천하는 대표적인 기술 스택
- ❖ LAMP 스택의 목적은 웹 애플리케이션을 쉽게 만들고 관리하는 것
 - 각 기술이 함께 작동하며, 웹 서비스의 중요한 역할 수행

□ **Linux**: 안정적이고 확장 가능한 운영 체제 → LAMP의 다른 기술은 모두 Linux 위에서 실행

□ **Apache**: 웹 서버로, 사용자의 요청을 처리하고 웹 페이지를 보여줌 → HTTP 프로토콜을 사용해 웹과 통신

□ **MySQL**: 데이터를 저장하고 관리하는 데이터베이스 → 대규모 데이터 처리가 필요한 애플리케이션에 사용

□ **PHP**: 동적인 웹 콘텐츠를 만드는 스크립팅 언어 → 사용자 요청에 따라 실시간으로 웹 페이지를 생성하고 데이터를 처리



2. 오픈소스 SW의 역사

9 LAMP 스택

- **오픈 소스 생태계:** LAMP 스택의 모든 기술은 무료로 제공
 - 개발자는 비용을 절약하면서 고품질의 소프트웨어 사용
 - 상업 소프트웨어와 경쟁할 수 있는 강력한 대안
- **커뮤니티 기반 개발:** LAMP의 모든 기술은 전 세계 개발자들이 기여하며 발전
 - 보안성과 성능이 지속적으로 유지
 - 문제 해결과 기능 확장이 쉬움 (오픈 소스 커뮤니티)
- **유연성과 확장성:** LAMP 스택은 필요에 맞게 조정 가능
 - 예) MySQL 대신 PostgreSQL 사용, PHP 대신 Python 사용
 - 개발자는 프로젝트에 맞는 최적의 기술 선택 가능
- **비용 절감:** 모든 기술이 무료로 제공되어 비용 절감



2. 오픈소스 SW의 역사

10 Microsoft와 오픈소스

❖ 할로윈 문서(Halloween Documents)

- 마이크로소프트가 오픈 소프트웨어와 리눅스에 대해 논의한 비밀 메모
- 1998년, 에릭 레이먼드가 처음으로 문서를 유출 및 공개
- 문서 내용:
 - ✓ 오픈 소스 소프트웨어, 특히 리눅스가 마이크로소프트의 시장 지배에 큰 위협이라고 평가
 - ✓ 오픈 소스가 기술적으로 경쟁할 수 있음을 인정
 - ✓ 마이크로소프트는 오픈 소스의 성장을 방해할 전략을 세움
- 마이크로소프트는 문서의 진본임을 인정했고, 이후에도 추가 문서가 유출

❖ 마이크로소프트의 당시 배경

- ❖ 상업적 소프트웨어 기업: 마이크로소프트는 상업적 소프트웨어 시장의 지배적인 기업이었음
- ❖ 주요 수익: Windows 운영 체제와 Office 제품군으로 수익 창출
 - ❖ 오픈 소스 소프트웨어는 무료로 제공되어, 마이크로소프트의 상업적 모델에 위협
 - ❖ 리눅스는 서버 시장에서 빠르게 성장하며, 마이크로소프트의 윈도우 서버 시장을 잠식할 가능성 있음
- ❖ 시장 방어 전략: 마이크로소프트는 상업적 소프트웨어 모델을 유지하기 위해, **오픈 소스의 성장을 억제**하려 함



VS



2. 오픈소스 SW의 역사

10 Microsoft와 오픈소스

❖ FUD(Fear, Uncertainty, Doubt) 전략

- 마이크로소프트가 오픈 소스 소프트웨어에 대한 부정적 인식을 조성하기 위한 전략
- ➔ 경쟁 제품에 대한 신뢰를 떨어뜨리고, 자사 소프트웨어에 대한 신뢰를 높이는 것이 목적
- 두려움(Fear): 오픈 소스가 안정성과 보안 문제를 일으킬 수 있다고 주장
- 불확실성(Uncertainty): 오픈 소스의 법적 문제나 호환성을 강조
- 의심(Doubt): 오픈 소스가 장기적인 유지보수나 지원을 받을 수 없다는 의문 제기

❖ FUD 전략의 결과

- 마이크로소프트는 오픈 소스에 대한 부정적 인식을 조성하려 했으나, 실패
- 글로벌 개발자 커뮤니티의 지속적인 성장과 오픈 소스 소프트웨어 발전을 막을 수 없었음
- 현재, 리눅스와 오픈 소스는 전 세계적으로 널리 사용되는 강력한 기술 생태계로 자리 잡음



VS



2. 오픈소스 SW의 역사

11 Microsoft의 변화와 Shared Source

❖ 마이크로소프트의 오픈 소스 전환

- 마이크로소프트는 이후 오픈 소스 커뮤니티와 협력하는 방향으로 전환
- GitHub 인수와 같은 오픈 소스 친화적 정책 추진

Organizations with the most open source contributors

 Microsoft	16,419	 atom	9,698
 facebook	15,682	 FortAwesome	9,617
 docker	14,059	 elastic	7,220
 angular	12,841	 Apache	6,999
 google	12,140	 npm	6,815

❖ Shared Source Initiative (SSI)

- 2001년 시작된 소스 코드 공개 프로그램
- 개인과 기업이 참고 또는 보안 검토를 위해 마이크로소프트의 소스 코드에 접근 가능
- 5가지 라이선스 제공:
 - ✓ Microsoft Public License와 Microsoft Reciprocal License는 오픈 소스 라이선스로 승인됨
 - ✓ 나머지 라이선스는 독점적으로, 마이크로소프트가 더 많은 통제를 유지
- 일부 제품에 맞춘 특정 라이선스도 사용 (예: Shared Source CLI License)



2. 오픈소스 SW의 역사

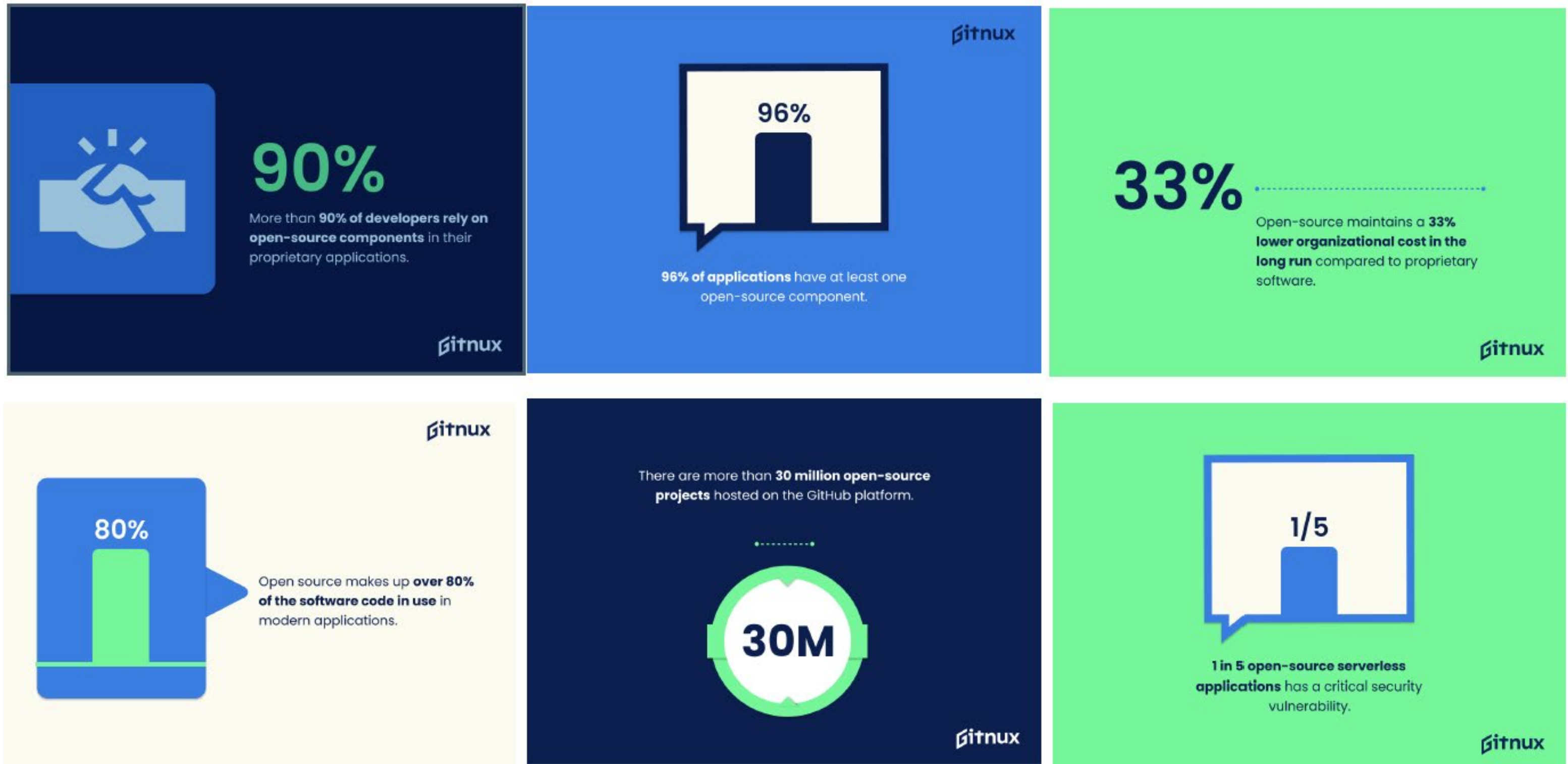
12 오픈소스 SW 현재 동향



2. 오픈소스 SW의 역사

12 오픈소스 SW 현재 동향

출처: [GITNEX MARKETDATA REPORT 2024](#)



2. 오픈소스 SW의 역사

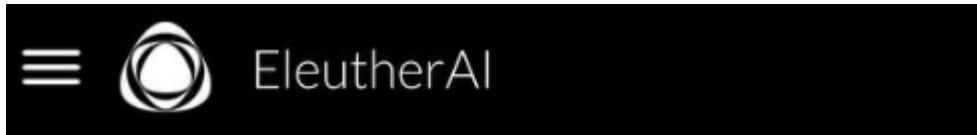
12 오픈소스 SW 현재 동향



- 해석 가능성 & 정렬(alignment) 문제 연구에 집중
- AI 기술의 잠재적 위험성에 대한 경고 → 투명성 강조
- AI 발전 방향이 소수의 대기업에 독점되어서는 안되며,
• 누구나 연구할 수 있는 환경을 만드는 것이 목표
- 소스코드와 연구성과 공개 → 오픈소스 생태계에 기여

2. 오픈소스 SW의 역사

12 오픈소스 SW 현재 동향



About Us

EleutherAI (/iˈluθər ei. aɪ/) is a grassroots collection of researchers working to open source AI research. Founded in July of 2020, our flagship project is GPT-Neo, a replication of OpenAI's massive 175B parameter language model, GPT-3. Our Discord server is open and welcomes contributors of all backgrounds.

Links

- [Github Information Page](#)
- [Check out the dataset code](#)
- [Check out the model code](#)
- [Watch the models train](#)

Pythia Scaling Suite

Pythia is the first LLM suite designed specifically to enable scientific research on LLMs. To learn more see <https://github.com/EleutherAI/pythia>

▲ Upvote 31

👤 +21

Pythia: A Suite for Analyzing Large Language ...

📄 Paper • 2304.01373 • Published Apr 4, 2023 • △ 9

Note The paper introduced the Pythia model suite

👤 EleutherAI/pythia-14m

📄 Text Generation • 📊 0.0B • Upda... • 📄 46.3k • ❤️ 23

Note This model was trained after the paper was released.

👤 EleutherAI/pythia-70m

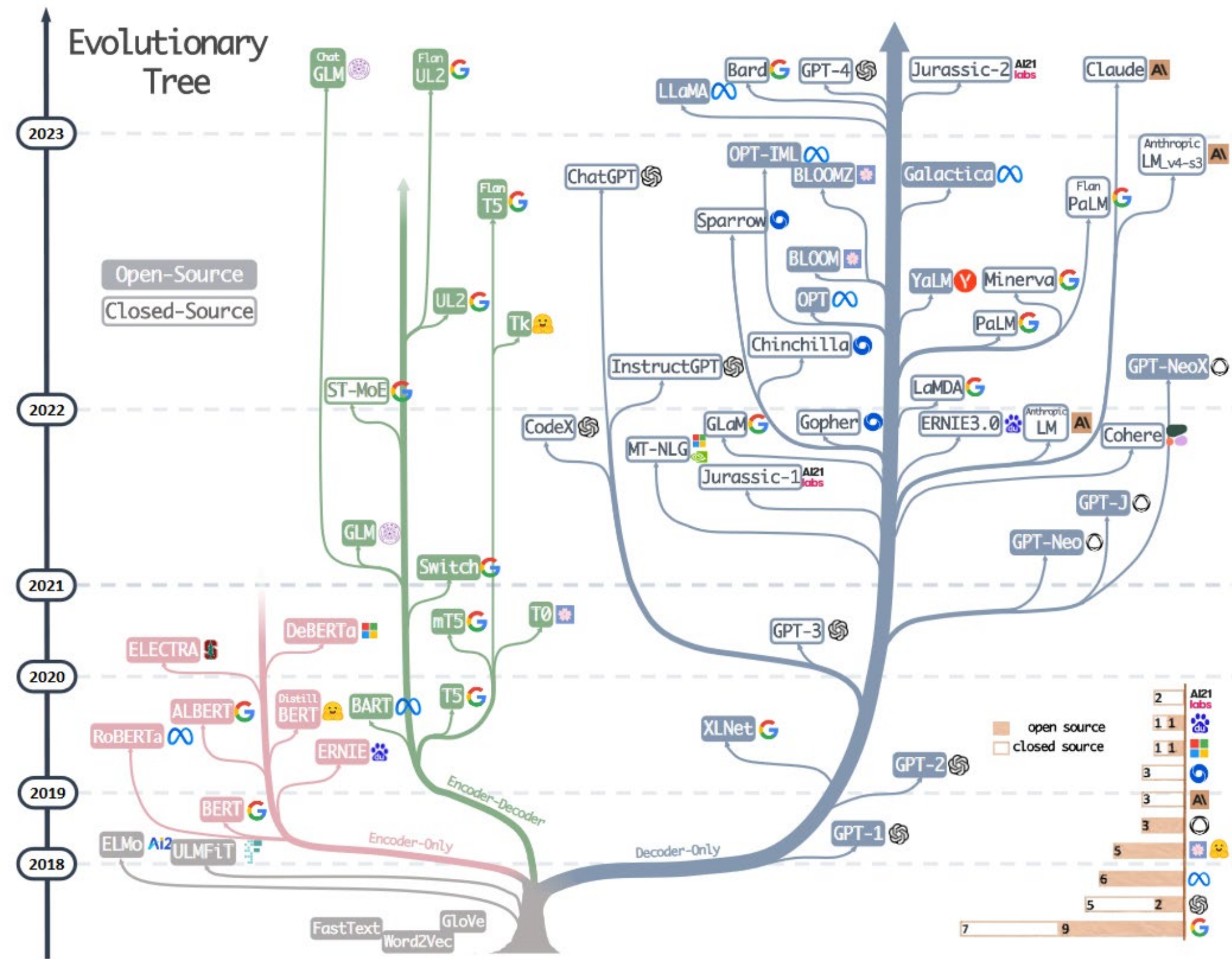
📄 0.1B • Updated Nov 22, 2023 • 📄 152k • ❤️ 73

Note This model was trained after the paper was released.

- 데이터셋 공개
- 체크포인트 공개
- 다양한 모델 사이즈 공개
- 학습과정 공개
- 모델 아키텍처 공개

2. 오픈소스 SW의 역사

12 오픈소스 SW 현재 동향



("Harnessing the Power of LLMs in Practice", Yang et al., 2023)

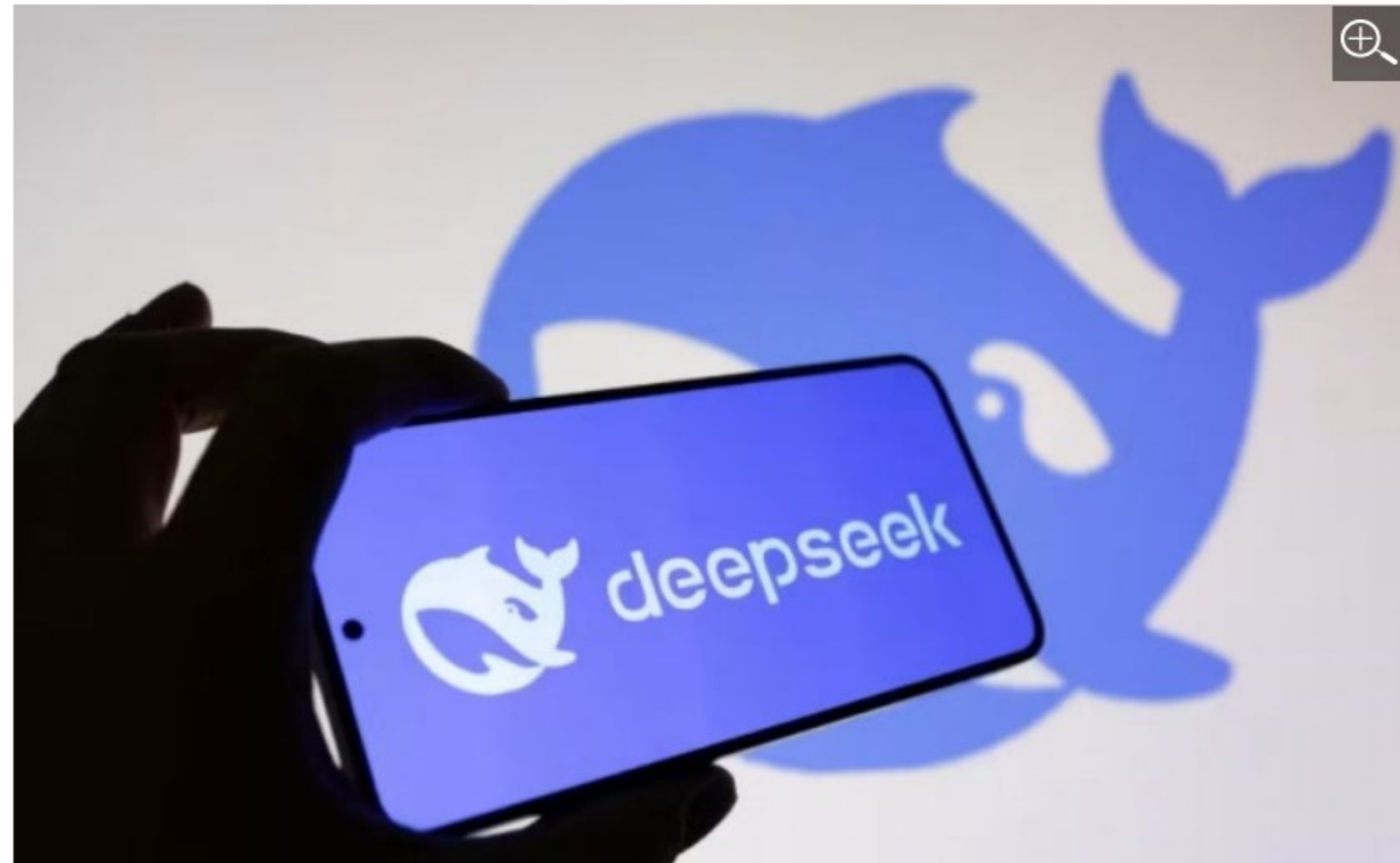
2. 오픈소스 SW의 역사

12 오픈소스 SW 현재 동향

[심층분석] 중국, 오픈소스 AI로 미국 기술 주도권에 도전

박정환 기자 입력 2025-08-14 01:35

알리바바·딥시크 등 무료 모델로 세계 시장 공략...美, 독점적 지위 흔들리자 오픈소스로 맞불
성능·현지화 내세워 아시아 시장서 각광...'다윈식 생존경쟁' 거쳐 거대 기업 탄생 예고



중국의 인공지능(AI) 딥시크. 중국은 특정 기업이 기술을 독점하는 '폐쇄형' 전략을 고수해 온 미국과 달리, 소스코드를 전면 공개하는 '오픈소스'를 앞세워

(“Harnessing the Power of LLMs in Practice”, Yang et al., 2023)

2. 오픈소스 SW의 역사

12 오픈소스 SW 현재 동향

산업일반

엔트로픽, 소송 합의금으로 2조 지급..."AI 저작권 인정 첫 사례"

임대준 기자

업데이트 2025.09.06 10:03 ▾ | 댓글 2



1. 오픈소스 SW란?

2 OX 퀴즈

문장	정답
GNU와 Linux는 완전히 같은 것이다	
시장 모델은 개발과정을 공개하고 자주 배포한다	
성당 모델은 누구나 실시간으로 코드에 기여한다	
Microsoft는 처음부터 계속 오픈소스에 우호적이었다	

오픈소스 소프트웨어 개발

한림대학교

3. 오픈소스 SW의 개발

1 오픈소스SW의 생태계

❖ 오픈소스SW 생태계 이해

- 오픈소스SW를 선택하기 전에 생태계를 이해하는 것이 중요함
- 오픈소스SW 생태계는 크게 생산자, 공급자, 소비자로 나뉨
- 이 생태계는 다양한 참여자들이 상호작용하며, 오픈소스SW의 개발, 배포, 사용을 촉진함



3. 오픈소스 SW의 개발

1 오픈소스SW의 생태계

1) 생산자

- 오픈소스SW 커뮤니티: 생태계에서 허브 역할을 하며, 개발자들이 협력하여 소프트웨어를 발전시킴
- 오픈소스SW 재단: 커뮤니티와 상용 오픈소스SW 벤더 간의 공동 작업을 지원하며, 프로젝트 관리 역할을 담당
- 오픈소스SW 기여자: 오픈소스SW를 개발하고 문제를 해결하는 개발자 및 참여자들

2) 공급자

- 소프트웨어 인프라 공급자: 오픈소스SW를 지원하는 기술적 인프라를 제공
- 교육 및 컨설팅 기업: 오픈소스SW 관련 교육과 컨설팅 서비스를 제공하여 비즈니스 생태계 조성
- 기술 지원 기업: 오픈소스SW 커뮤니티와 협력하여 소프트웨어의 유지 및 기술 지원을 제공하는 기업

3) 소비자

- 일반 사용자: 오픈소스SW를 사용하고, 커뮤니티 활동을 통해 소프트웨어 개선에 기여
- 오픈소스SW 유료 사용자: 유료 서비스로 추가적인 지원을 받는 사용자 또는 조직

3. 오픈소스 SW의 개발

2 오픈소스SW 커뮤니티

❖ 오픈소스SW 커뮤니티의 구성

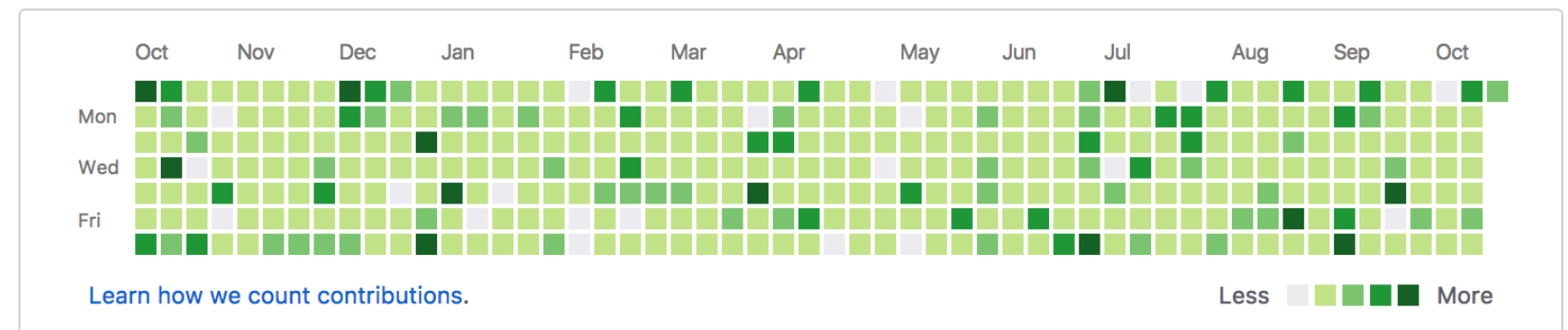
- 양파 구조로 비유됨
 - 프로젝트 리더(Project Leader, PL): 오픈소스SW 프로젝트의 최종 의사 결정을 내림
 - 핵심 기여자: 프로젝트 경험이 가장 많은 실력자로, 커뮤니티 구성원들을 지도하거나 멘토링함
 - 기여자: 프로젝트에 기여하는 일반적인 개발자들
 - 신입 기여자: 처음으로 기여하는 초보 개발자들
 - 사용자: 소프트웨어를 사용하는 일반 사용자들

Contributors



1,387 contributions in the last year

Contribution settings ▾



3. 오픈소스 SW의 개발

3 오픈소스SW 재단

❖ 오픈소스SW 재단의 이해

- 오픈소스SW 생태계에는 다양한 재단이 존재하며, 기업들이 **신뢰할 수 있는 비영리 단체**를 통해 협력할 수 있도록 함
- **재단 설립**은 경쟁 관계에 있는 기업들이 협력할 수 있는 **중립적인 공간**을 제공
- 재단을 통해 기업들은 공동으로 **오픈소스 프로젝트**를 추진하고, 기술적 지원 및 법적 보호를 받음

○ 1.OSI (Open Source Initiative) - <http://opensource.org/>

- 1998년 Netscape에서 영감을 받아 설립됨
- 오픈소스 소프트웨어 사용을 장려하고, **OSI 라이선스 검토와 승인**을 담당
- **Open Source Definition (OSD)**을 관리하여 오픈소스의 기본 원칙을 유지

○ 2. Free Software Foundation - <https://www.fsf.org>

- 1985년, 리처드 스톨만에 의해 설립된 재단.
- **자유 소프트웨어**의 생산과 배포를 장려하며, 컴퓨터 소프트웨어를 누구나 자유롭게 수정하고 공유할 수 있도록 촉진
- 자유 소프트웨어 운동의 주요 단체로, 사용자에게 소프트웨어의 자유를 제공하는 데 중점을 둠

3. 오픈소스 SW의 개발

3 오픈소스SW 재단

- **3. Linux Foundation** - <https://linuxfoundation.org>
 - 리눅스의 성장을 지원하는 기술 컨소시엄
 - 리눅스 외에도 다양한 오픈소스 프로젝트를 지원하며, **상업적 지원 및 교육, 인증 프로그램** 등을 제공.
 - 오픈소스 생태계 전반에서 중요한 역할을 하며, 리눅스뿐만 아니라 여러 다른 프로젝트의 성공적인 운영을 돕고 있음
- **4. Apache Software Foundation** - <http://apache.org>
 - 1993년, Apache HTTP Server 개발자들로 시작하여 분산 개발자 커뮤니티로 발전
 - 아파치 프로젝트에서 개발된 소프트웨어는 Apache 라이선스 하에 배포됨
 - 재단은 법적 보호를 제공하며, ApacheCon 콘퍼런스를 매년 개최해 개발자들과 기술 전문가들이 교류할 수 있는 장을 제공
- **5. Cloud Foundry Foundation** - <https://www.cloudfoundry.org/foundation/>
 - 클라우드 파운드리 오픈소스 프로젝트의 인식 확대와 채택을 지원
 - EMC, HP, IBM, 인텔, SAP 등 여러 기업들이 공동 참여
 - 글로벌 파트너들과 함께 기여자 커뮤니티 성장 및 프로젝트 성공을 위해 전략과 조치를 조정

3. 오픈소스 SW의 개발

4 오픈소스SW 라이선스 (간략)

❖ 오픈소스SW라이선스 개요

- 오픈소스SW는 저작권법에 따라 보호받음
- 소스 코드가 공개되어 누구나 자유롭게 사용, 수정, 배포 가능
- 오픈소스SW 라이선스는 사용자가 지켜야 할 의무 사항을 명시
- 의무 사항을 지키지 않으면 저작권법 위반이 됨

❖ GPL (GNU General Public License) 라이선스

- 오픈소스SW 사용 시, 모든 코드를 공개해야 하는 의무 있음
- 상용 소프트웨어에서 코드 공개 없이 사용 시 저작권법 위반으로 형사처벌 가능



3. 오픈소스 SW의 개발

4 오픈소스SW 라이선스 (간략)

❖ 오픈소스SW 라이선스 구분

□ Permissive 라이선스

- 고지 의무만 지키면 자유롭게 사용 가능
- 예) MIT, BSD, Apache 라이선스

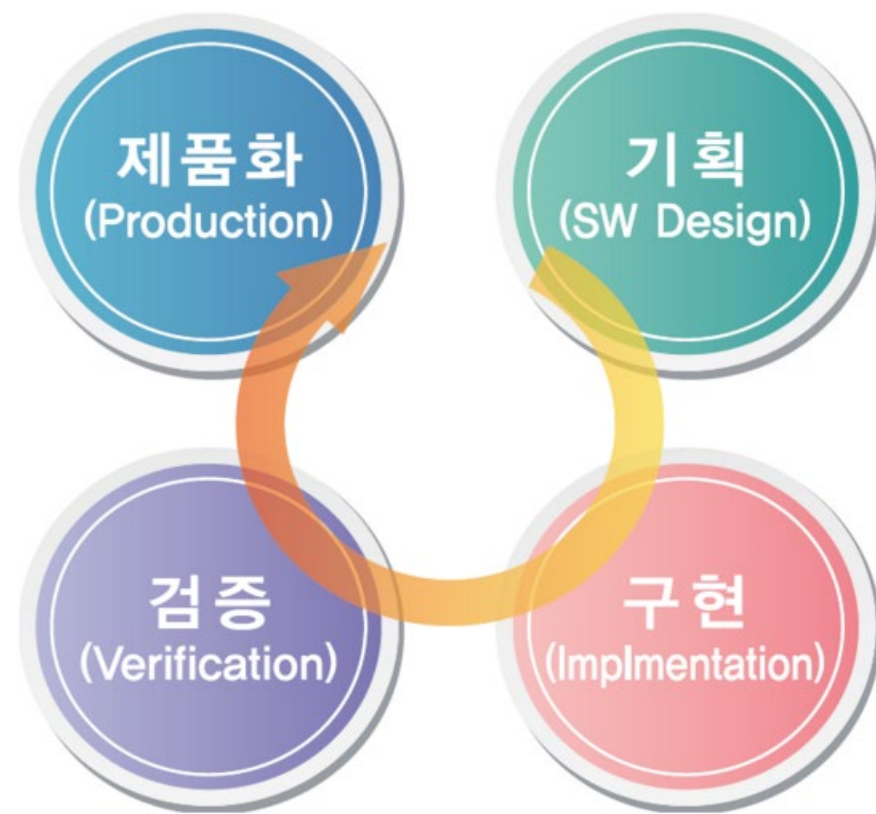
□ Copyleft 라이선스

- 고지 의무 + 코드 공개 의무 있음
- 동일한 조건으로 소프트웨어를 배포해야 함
- 예) GPL, LGPL, AGPL, MPL 라이선스



3. 오픈소스 SW의 개발

5 오픈소스SW 개발 관리 프로세스



□ 1. 기획 단계

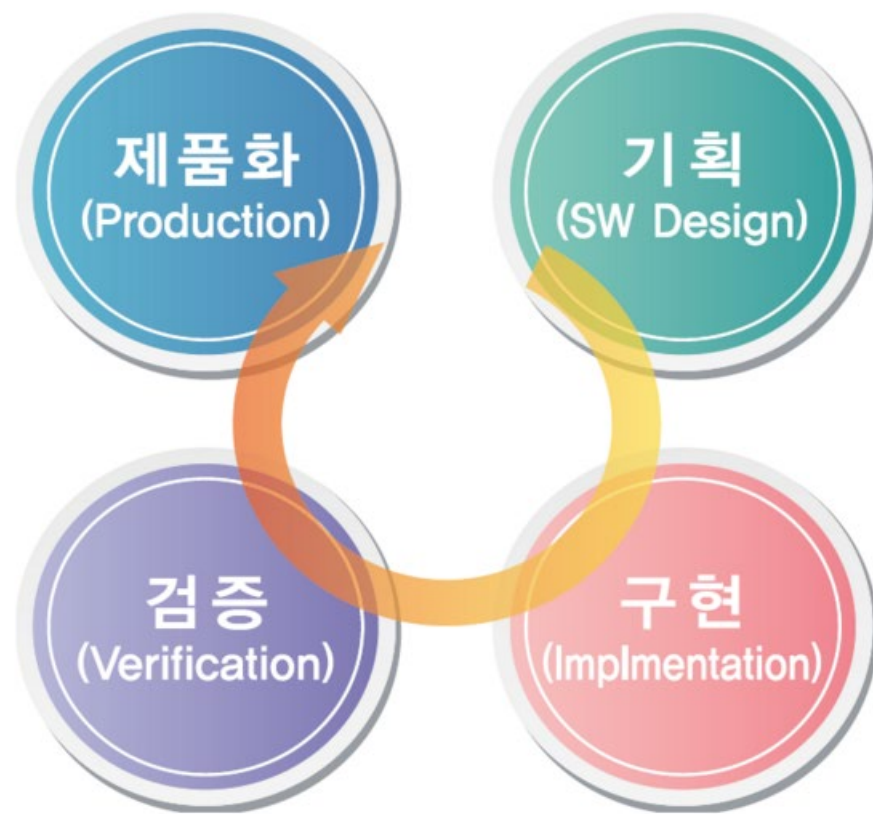
- 사용할 오픈소스SW와 라이선스 규정 파악
- 소스코드 공개 여부 및 라이선스 양립성 확인
 - 예) GPL과 MIT 라이선스를 함께 사용할 수 있는지 검토

□ 2. 구현 단계

- 필요한 오픈소스SW 검색 및 산출물 수집
- 사용한 SW 목록 작성 및 라이선스 규정 준수 확인
 - 예) 사용한 라이브러리 목록 작성 및 각 라이선스의 의무사항 기록

3. 오픈소스 SW의 개발

5 오픈소스SW 개발 관리 프로세스



□ 3. 검증 단계

- 오픈소스 라이선스가 적절히 사용되었는지 검증
- 예) FOSSology 같은 오픈소스 검증 도구로 사용된 라이선스 검토
- 검증 시스템이나 스트링 검색으로 소스코드 확인
- 예) grep 명령어 사용

□ 4. 제품화 단계

- 라이선스별 준수사항 제품에 반영
- 소스코드 공개 여부 결정 및 제공 방법 설정
- 예) 제품과 함께 GitHub 리포지토리 링크 제공으로 소스코드 공개

3. 오픈소스 SW의 개발

6 오픈소스SW 개발 특징

오픈소스 SW 특징	설명
상향식 개발 (Bottom-Up Development)	다수의 개발자가 설계에 참여하며, 사용자 피드백을 반영해 지속적으로 수정 및 개선
초기 배포, 빈번한 배포 (Release early, Release often)	빠른 배포와 피드백을 통해 기능을 빠르게 발전시키고, 커뮤니티 요구를 반영
동료 검토	다양한 개발자가 코드 리뷰를 통해 품질을 개선하고, 오류를 최소화
소규모, 점진적 개선	소규모 기능 추가와 사소한 수정으로 점진적으로 프로젝트를 발전
보안 취약점 표시	보안 문제를 빠르게 파악하고 즉각적으로 수정 가능 (오픈소스 커뮤니티)
지속적 품질향상	동료 검토와 피드백을 통해 소프트웨어의 지속적인 품질 향상
테스트 자동화	대규모 프로젝트에서 자동화된 테스트 환경을 통해 신속한 품질 관리 가능
전 과정 사용자 참여	사용자들이 개발 전 과정에 참여하여 기능 개선과 버그 수정에 기여

3. 오픈소스 SW의 개발

7 오픈소스SW 가치 기준 및 판단 이유

1. 동포함성 (Inclusivity):

- 누구나 쉽게 접근 가능
- 개발자뿐만 아니라 사용자도 소스코드를 수정하고 기여할 수 있음

2. 투명성 (Transparency):

- 소스코드가 공개되어 모든 개발 과정이 투명하게 공개됨
- 개발자, 사용자는 제품의 신뢰성을 확인할 수 있음

3. 안전성 (Security):

- 다양한 개발자들이 코드를 검토하고 문제를 발견해 신속히 해결
- 보안 취약점이 빠르게 발견되고 수정됨

4. 유연성 (Flexibility):

- 다양한 환경에서 사용 가능
- 필요에 따라 쉽게 수정하고 기능을 추가할 수 있음

5. 경제성 (Economy):

- 대부분의 오픈소스 SW는 무료로 제공됨
- 상용 SW에 비해 개발 비용을 크게 절감할 수 있음

6. 독립성 (Independence):

- 특정 회사나 제품에 종속되지 않음
- 특정 벤더나 제품의 독점적 요구 없이 자유롭게 사용 가능

7. 협력/협업 (Collaboration):

- 다양한 커뮤니티가 협력하여 더 나은 SW 개발 가능
- 프로젝트에 기여할 수 있는 기회가 많고, 전 세계의 개발자들과 협업 가능

3. 오픈소스 SW의 개발

8 오픈소스SW의 장단점

❖ 오픈소스의 장점?

3. 오픈소스 SW의 개발

8 오픈소스SW의 장단점

❖ 오픈소스의 장점

- **낮은 개발 비용:** 오픈 소스 소프트웨어는 무료로 사용할 수 있어 비용이 절감
- **빠른 기술 지원:** 전 세계 개발자 커뮤니티가 기술 지원을 제공해 문제 해결이 빠름
 - 프리-오픈소스 소프트웨어 혁명의 역사, 크리스토퍼 토씨
- **유연한 개발:** 소스 코드가 공개되어 있어 필요에 맞게 소프트웨어를 수정하고 맞춤화 가능
- **오픈 포맷과 프로토콜:** 표준화된 오픈 포맷을 사용하여 다른 소프트웨어와 호환성 높음
- **신뢰성과 안정성:** 많은 사용자와 개발자들이 검토하고 테스트하여 신뢰성과 안정성이 높음
- **강력한 네트워킹 지원:** 다양한 네트워크 및 플랫폼에서 자유롭게 사용할 수 있음

3. 오픈소스 SW의 개발

8 오픈소스SW의 장단점

❖ 오픈소스의 단점?

3. 오픈소스 SW의 개발

8 오픈소스SW의 장단점

❖ 오픈소스의 단점

- **빈약한 문서:** 사용자 가이드나 기술 문서가 부족할 수 있음
- **불확실한 개발 로드맵:** 개발 계획이 명확하지 않아서 향후 업데이트나 기능 추가가 예측하기 어려움
- **지적 재산권 문제:** 오픈 소스 라이선스나 코드 사용과 관련한 지적 재산권 문제가 발생 가능
- **기술 지원 부족:** 상용 소프트웨어에 비해 공식적인 기술 지원이 부족하여 초심자 난이도 ↑
- **기능 통합의 어려움:** 상업적 소프트웨어에 비해 여러 기능이 통합되지 않아 사용자 편의성 ↓

오늘은 여기까지.

Thank you for listening.

REFERENCES

- 프리-오픈소스 소프트웨어 혁명의 역사, 크리스토퍼 토씨
- 성당과 시장: 우연한 혁명으로 일어난 리눅스와 오픈소스에 대한 생각, 에릭 레이먼드
- 2024 기업 오픈소스 SW 거버넌스 가이드, 정보통신산업진흥원
- 형상관리 툴인 Git의 활용, 한림대SW대학